

How the App Talks to the Server

Every page, every API call, every parameter

Project Karma (Antra CRM field app) — Flutter / GetX

Source generated from the codebase, branch `main`

Date 2 September 2026

Companion ROLE-ACCESS-DOCUMENTATION.pdf

Who this is for: anyone. Developer, tester, BA, new joiner, or support. Sections 1 to 5 need no Flutter knowledge. Section 7 onward is the reference you look things up in.

If you have 2 minutes, read section 2. If you're debugging one screen, jump to section 7 and find it by name.

Table of contents

#	Section	Read it when
1	What this app is	You're new
2	How it talks to the server — the one idea	Always. Start here.
3	A complete example, start to finish	You want to see it work once
4	Where the code lives	You need to find a file
5	The numbers	You want the size of things
6	The two API layers	You're writing a new call
7	Every page and its API calls	You're debugging a screen
8	Dictionary of all 88 actions	You saw an ACTION and don't know it
9	The 6 payload formats	Your request is failing
10	Session globals	You're wondering where <code>UID</code> comes from
11	How to add a new screen	You're building something
12	Glossary	A word here is unfamiliar
13	Engineering notes	You're planning improvements

1. What this app is

Karma is a **field CRM app for Antra**. Sales and support staff use it on their phones to:

- look up customers and their contact people,
- book customer visits, check in and check out of them,
- raise and track support tickets,
- record leads and opportunities,
- claim travel expenses and apply for leave,
- view targets, rankings and outstanding payments.

It is built in **Flutter** and uses **GetX** for state management and navigation.

It has **no local database of its own** for business data. Almost every screen is a live view of the server. Open a screen → it calls the server → it shows what comes back.

2. How it talks to the server — the one idea

This is the most important section in the document. Everything else follows from it.

There is only one server address

```
https://gateway.tallyhelp.com/crm/UpdateData.aspx
```

Every screen in the app calls this same address. There are no /customers , /tickets , /leads paths. One URL, for everything.

So how does the server know what you want?

You tell it in a query parameter called ACTION .

Think of it like ordering at a single counter. You always walk to the same counter. What changes is what you ask for (ACTION) and the slip you hand over (DATA).

You want	You say ACTION=	You hand over DATA=
The staff directory	ANTRAUSER	(nothing)
My customer list	GETDATAPOINT	vinod
One customer's info	GETDPINF	{"DPID":"56789"}
Create a ticket	ANTRATICKET	the ticket details

Every request looks exactly like this

```
https://gateway.tallyhelp.com/crm/UpdateData.aspx?DATA=<payload>&ACTION=<name>&ENCKEY=<key>&SRC=KARMA&ISNEW=YES
```

what you're sending

what you want

your session key (proves who you are)

always "KARMA" (tells the server it's this app)

always "YES" (new API version)

Five parameters. Three of them (ENCKEY , SRC , ISNEW) are the same on every single call. Only DATA and ACTION change.

That's the whole protocol

- 88 different ACTION values exist across the app — see section 8 for what each one means.
- 101 places in the code make a call.
- Almost all are GET requests. Only 5 are POST.
- The response is JSON, usually shaped { "records": [...] } .

Once you understand ACTION + DATA , you understand every network call in this app.

One thing that will confuse you

DATA is not always JSON. The backend accepts six different formats, and different screens use different ones — sometimes for the same kind of data:

DATA=	← empty
DATA=vinod	← a plain value
DATA=vinod 56789	← pipe-separated
DATA=77~2~Followed up~1234~56789	← tilde-separated (only one screen does this)
DATA={"UID":"1234"}	← JSON
DATA=%7B%22UID%22%3A%221234%22%7D	← URL-encoded JSON

This is historical, not intentional. Section 9 lists which screens use which. Check the format before you edit a call — this is the single most common source of bugs here.

3. A complete example, start to finish

Let's follow the **Contacts** screen from the moment a user taps it to the moment names appear on screen. Every screen in this app works this way.

Step 1 — The user taps "Contact List" in the drawer

`lib/Application/Drawer/DrawerWidget.dart`

```
tileWidget(  
  onTap: () {  
    Get.back();  
    Get.to(() => const Contacts()); // ← navigate to the Contacts screen  
  },  
  title: "Contact List",  
);
```

Step 2 — The screen is created, and it creates its controller

`lib/Application/Contacts/Contacts.dart:6-11`

```
class Contacts extends GetView<ContactsController> { // ← "my brain is ContactsController"  
  
  @override  
  Widget build(BuildContext context) {  
    Get.put(ContactsController()); // ← create the brain  
    return Scaffold(...);  
  }  
}
```

`GetView<ContactsController>` gives this screen a free `controller` property. `Get.put(...)` builds the controller — **and that is what triggers the API call.**

Step 3 — Creating the controller automatically fires `onInit`

`lib/Controller/contactsController.dart:13-16`

```
@override  
void onInit() {  
  getData(); // ← GetX calls onInit automatically. This starts the network request.  
  super.onInit();  
}
```

Step 4 — The controller calls the server

`lib/Controller/contactsController.dart:26-52`

```
Future<void> getData() async {  
  isLoading.value = true; // 1. show the spinner  
  hasError.value = false;  
  try {  
    final value = await Api().fetchApi(action: "ANTRAUSER"); // 2. THE NETWORK CALL  
  
    if (value != null && value.success) {  
      contactList.value = value.data['records']; // 3. save the results  
      allContacts.value = value.data['records']; // (a second, untouched copy)  
    } else {  
      hasError.value = true; // 4. or flag the failure  
    }  
  } catch (e) {  
    hasError.value = true;  
  } finally {  
    isLoading.value = false; // 5. hide the spinner, always  
  }  
}
```

Step 5 — What actually goes over the wire

`Api().fetchApi(action: "ANTRAUSER")` builds this URL:

```
GET https://gateway.tallyhelp.com/crm/UpdateData.aspx?DATA=&ACTION=ANTRAUSER&ENCKEY=ABC123XYZ&SRC=KARMA&ISNEW=YES
```

Note `DATA=` is empty — this call needs no input. It just says "give me the staff list".

Step 6 — What comes back

```
{
  "records": [
    { "NAME": "Vinod Sharma", "DESIGNATION": "Sr. Executive",
      "TEAMNAME": "West Zone", "teamOwner": "Rakesh Patel" },
    { "NAME": "Anita Desai", "DESIGNATION": "Support Lead",
      "TEAMNAME": "Support", "teamOwner": "Meera Joshi" }
  ]
}
```

Step 7 — The screen redraws by itself

`lib/Application/Contacts/Contacts.dart:16–47`

```
body: Obx(() => Column(          // ← Obx watches the controller's values
  children: [
    SearchWidget(...),
    AsyncStateView(              // ← this widget picks what to show
      isLoading: controller.isLoading.value,      // spinner?
      hasError: controller.hasError.value,        // error + retry button?
      isEmpty: controller.contactList.isEmpty,    // "nothing here"?
      onRetry: controller.getData,
      child: ListView.builder(                     // or the actual list
        itemCount: controller.contactList.length,
        itemBuilder: (context, index) =>
          tileWidget(controller.contactList[index]),
      ),
    ),
  ],
),
)),
```

Nobody calls "refresh the screen". `contactList` is an **observable** (`RxList`). When the controller assigns to it, `Obx` notices and rebuilds automatically.

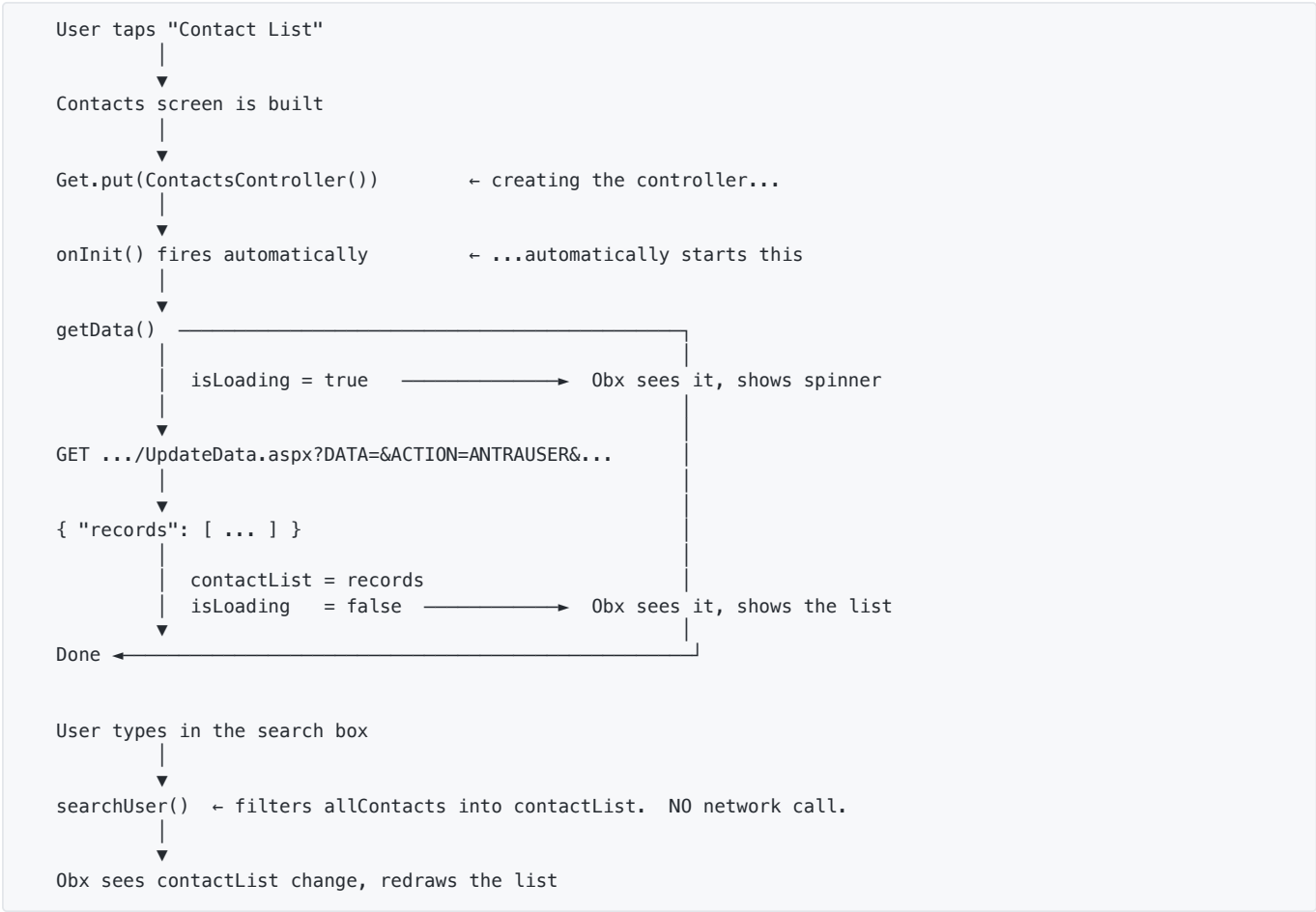
Step 8 — Typing in the search box does NOT call the server

`lib/Controller/contactsController.dart:68–80`

```
searchUser() {
  if (search.value.trim().isEmpty) {
    contactList.value = List.from(allContacts); // restore the full list
  } else {
    // rank matches: NAME beats teamOwner beats DESIGNATION beats TEAMNAME
    final ranked = allContacts
      .map((e) => MapEntry(e, _matchPriority(e, search.value)))
      .where((entry) => entry.value > 0)
      .toList()..sort((a, b) => a.value.compareTo(b.value));
    contactList.value = ranked.map((e) => e.key).toList();
  }
}
```

This is why the controller keeps **two** lists. `allContacts` is the untouched server response; `contactList` is what's on screen. Search filters one into the other — **entirely offline, no network call**.

The whole flow on one page



Every screen in this app follows this exact shape. Once you can read Contacts, you can read all 84 screens.

4. Where the code lives

The two-file rule

Every feature is **exactly two files**:

lib/Application/Contacts/Contacts.dart

lib/Controller/contactsController.dart

← THE SCREEN. How it looks.

← THE CONTROLLER. What it does and knows.

	The Screen	The Controller
Folder	lib/Application/<Feature>/	lib/Controller/
Contains	Scaffold, AppBar, colours, padding, lists, cards	API calls, Rx state, filtering, validation
Never contains	Api , Dio , http	any widget
Rule of thumb	"If I change this, the screen looks different"	"If I change this, the data is different"

That test settles almost every "where does this code go?" question.

The full folder map

lib/

- main.dart
- Application/
 - Contacts/
 - Dashboard/
 - Ticket/
 - ...
- Controller/
 - contactsController.dart
 - dashboardController.dart
 - ...
- Services/
 - Api.dart
 - Apis.dart
 - webApis.dart
 - db_helper.dart
 - SecureCredentials.dart
- Widgets/
 - AsyncStateView.dart
 - AppBarWidget.dart
 - SearchWidget.dart
 - ...
- Constants/
 - Library.dart
 - dataInfo.dart
 - colorConstant.dart
 - customTextStyles.dart
 - imageConstants.dart
- Route/AppRoutes.dart
- Bindings/initialBindings.dart
- Models/
- core/ domain/ test/

App starts here

★ ALL SCREENS – 93 files, 29 feature folders
one folder per feature

★ ALL LOGIC – 62 files, flat (no subfolders)

★ THE NETWORK LAYER – 7 files
the Dio wrapper – use this one
older http wrapper – being phased out
the base URLs
local SQLite (small use)
saved login

Shared UI pieces – 14 files
loading / error / empty / content switcher

5 files
★ THE BARREL – 158 exports (see below)
★ session globals: userId, encKey, role...

route name constants

4 app-start controllers

only DeviceInfo + UserInfo

← EMPTY. Leftover scaffolding.

Why you only ever see one import

Most files start with just:

```
import 'package:karma/Constants/Library.dart';
```

Library.dart is a **barrel file** — it re-exports 158 things (Flutter, GetX, every controller, every widget, all constants). Importing it gives a screen everything at once.

Consequence: if you add a new controller or widget, add an **export** line to **Library.dart** or other files won't see it.

5. The numbers

Thing	Count
Files in lib/Application/	93
Actual screens (files with a Scaffold)	84
Helper files there that are not screens	9
Controllers	62
Controllers that make network calls	50
Distinct ACTION values	88

Thing	Count
Places in the code that call the server	101
Shared widgets	14
Screens using the shared <code>AsyncStateView</code>	20
Screens with role-based hiding	~14 (see the role doc)

The 9 files in `Application/` that are **not** screens — they are themes, shared widgets and one model:

`AGH/AGH.dart` · `AGH/AGHTheme.dart` · `AGH/AGHWidgets.dart` · `Dashboard/model/company_model.dart` · `Drawer/DrawerWidget.dart` · `Escalation/EscalationFromTicket.dart` · `Escalation/EscalationTheme.dart` · `Escalation/EscalationWidgets.dart` · `Utilities/Utilities.dart`

6. The two API layers

There are **two** ways to call the server in this codebase. This is not by design — the second is older and is being phased out.

Which should I use?

Always use `Api` (section 6.1). `Apis` exists only because parts of the app haven't been migrated yet. Never write new code against it.

6.1 `Api` — the current one · `lib/Services/Api.dart`

Built on **Dio**. This is where timeouts, logging and the slow-network warning live.

Method	HTTP	What it builds
<code>fetchApi()</code>	GET	<code>{base}DATA={data}&ACTION={action}&ENCKEY={key}&SRC=KARMA&ISNEW=YES</code>
<code>postApi()</code>	POST	URL <code>{base}ACTION={action}&DATA=</code> · body <code>{data}&ENCKEY={key}&SRC=KARMA&ISNEW=YES</code>
<code>postApi1()</code>	POST	URL <code>{base}ACTION={action}&DATA=&ENCKEY={key}&SRC=KARMA&ISNEW=YES</code> · body raw JSON

How to call it:

```
// no input
final value = await Api().fetchApi(action: "ANTRAUSER");

// a plain value
final value = await Api().fetchApi(data: "vinod", action: "GETDATAPOINT");

// JSON input
final value = await Api().fetchApi(
  data: json.encode({"DPID": "56789"}),
  action: "GETDPINF",
);

// a POST submit
final value = await Api().postApi1(
  data: json.encode({"categoryid": "12", "descr": "Printer down"}),
  action: "ANTRATICKET",
);
```

What you get back — always an `ApiResponse` :

```

class ApiResponse {
  bool success; // did it work?
  dynamic data; // the decoded JSON (or the raw string for submits)
  String? message; // error text, when the server sent one
}

```

What it does for you automatically:

Behaviour	Detail
Connect timeout	15 seconds
Receive timeout	5 minutes
Slow-network warning	a snackbar after 5 s, then a 10 s cooldown before it can show again
Debug logging	full request + response bodies in debug builds (<code>PrettyDioLogger</code>)
Error snackbar	shown automatically when <code>success == false</code>

⚠ Because `Api` already shows a snackbar on failure, a controller that *also* shows one makes the user see **two** errors for one problem. Set `hasError` and let the UI handle it.

6.2 `Apis` — the legacy one · `lib/Services/Apis.dart`

Built on the plain `http` package. No interceptors, no slow-network warning, a fixed 50-second timeout, and you decode the JSON yourself.

Still used by: **AGH** · **Products** · **Tickets** · **Profile** · **app-update**.

Method	Which URL	Shape
<code>sendData(data, action)</code>	<code>baseUrl</code>	<code>?DATA={data}&ACTION={action}</code> — no ENCKEY
<code>sendData1(data, action)</code>	<code>customerBaseUrl</code>	<code>?DATA={data}&ACTION={action}</code>
<code>sendData2(action)</code>	<code>customerBaseUrl</code>	<code>?ACTION={action}</code>
<code>sendData4(data, action)</code>	<code>baseUrl</code>	<code>?DATA={data}&ACTION={action}&ENCKEY={key}&SRC=KARMA</code>
<code>sendData5(data)</code>	<code>customerBaseUrl</code>	<code>?data={data}&ENCKEY={key}&SRC=KARMA</code>
<code>sendDataPost(action, body)</code>	<code>baseUrl</code>	POST <code>?ACTION={action}&DATA=&ENCKEY={key}&SRC=KARMA</code>

`sendData5` has no **ACTION** parameter at all. The action is smuggled in as the first segment of `data` :
`...CustomerApp.aspx?data=TICKETDETAILS|TCK00123|&ENCKEY=ABC123XYZ&SRC=KARMA` this is the action

6.3 The three base URLs · `lib/Services/webApis.dart`

```

static String rootUrl      = "https://gateway.tallyhelp.com";
static String baseUrl      = "$rootUrl/crm/UpdateData.aspx?";
static String customerBaseUrl = "$rootUrl/crm/CustomerApp.aspx?";

```

Name	Used for
<code>rootUrl</code>	profile images, file downloads
<code>baseUrl</code>	~95 % of all calls
<code>customerBaseUrl</code>	Products catalogue, ticket details

There is **no dev/staging switch**. A commented-out line (`http://192.168.1.22`) is the old local server. Everything points at production.

7. Every page and its API calls

How to read this section

Each entry looks like this:

`path/to/Screen.dart` — *ScreenName*

Controller: which controller drives it · **API calls:** N

#	Action	What it does	DATA sent

- **API calls: N** — how many distinct server calls this screen can make, including submits.
- **DATA sent** — with demo values filled in, so you can paste it into a browser.
- **submit** in the "What it does" column means it *writes* to the server.

Demo values used everywhere below:

```
UID / userId = 1234      UNAME / username = vinod
DPID         = 56789     CALLID          = 9001
OPPID        = 4501      TICKET           = TCK00123
ENROLLID     = EMP0042   ENCKEY          = ABC123XYZ
PID          = 7
```

7.1 Login & startup

`Application/SplashScreen/SplashScreen.dart` — *SplashScreen*

Controller: `AuthController` · **API calls:** 3

#	Action	What it does	DATA sent
1	<code>APPVERSION</code>	Asks the server for the latest app version, to prompt for an update	<code>10.0.0</code>
2	<code>LOGIN</code>	Signs the user in silently using saved credentials	URL-encoded JSON — see below
3	<code>REGISTERID</code>	Registers this phone + its notification token	JSON with FCM token, device id, model

```
GET {base}?DATA=10.0.0&ACTION=APPVERSION&ENCKEY=ABC123XYZ&SRC=KARMA&ISNEW=YES
```

`Application/Login/LoginPage.dart` — *LoginPage*

Controller: `LoginController` · **API calls:** 3

#	Action	What it does	DATA sent
1	<code>LOGIN</code>	Checks username + password, returns the user's profile and role flags	<code>{"USER","PWD","VERSION":"536","DEVICETYPE"}</code> URL-encoded
2	<code>REGISTERID</code>	Registers the device for push notifications	FCM token + device info
3	<i>(legacy</i> <code>Apis.sendData</code> <i>)</i>	Device registration follow-up	—

```
// DATA before encoding:
{"USER":"vinod","PWD":"pass@123","VERSION":"536","DEVICETYPE":"ANDROID"}

GET {base}?DATA=%7B%22USER%22%3A%22vinod%22%2C%22PWD%22%3A%22pass%40123%22%2C%22VERSION%22%3A%22536%22%2C%22DEVICETYPE%22%3A%22ANDROID%22%7D&ACTION=LOGIN&ENCKEY=&SRC=KARMA&ISNEW=YES
```

LOGIN is the most important call in the app. Its response populates every global in `DataInfo` — `UID` , `ENCKEY` , `DESCAT` , `ROLLID` , `TCID` , `TITLEID` . Every later request is built from those values. See section 10.

`Application/Onboarding/intro_screen.dart` · `Application/SyncProgress/sync_progress.dart`

API calls: 0 — static onboarding slides, and a progress UI driven by its caller.

7.2 Dashboard (home)

`Application/Dashboard/DashboardNew.dart` — *DashboardNew*

Controller: `DashboardController` (800+ lines) · **API calls: 11** ← the heaviest screen in the app

#	Action	What it does	DATA sent
1	<code>ACHIVEDTARGET</code>	Sales target achieved so far	<code>{"UID":"1234","PID":"7","UNAME":"vinod"}</code>
2	<code>ACHIVEDTARGET</code>	Same call again for a second widget	<i>(identical)</i>
3	<code>SUPDASHBOARD</code>	Support counts for the month	<code>{"UID","PID","UNAME","ENROLLID","REQTYPE":"MONTH"}</code>
4	<code>LEADDASHBOARD</code>	Leads this user has given out	<code>{"UID":"1234","LEADTYPE":"GIVEN"}</code>
5	<code>GETOPP</code>	Open opportunities	<code>{"UID":"1234","DPID":""}</code>
6	<code>GETTOPCUSTOMER</code>	Top customers widget	<code>{"UID":"1234"}</code>
7	<code>LEADDASHBOARD</code>	Lead counts	<code>{"UID":"1234"}</code>
8	<code>TEAMMEMBER</code>	Team list for the manager's dropdown	<code>1234</code>
9	<code>GETSUPRANK</code>	This user's support ranking	<code>{"UNAME":"vinod","REQTYPE":"MONTH"}</code>
10	<code>CALLBOOK</code>	Today's booked calls	<code>vinod</code>
11	<code>LEADDASHBOARD</code>	Refresh after an action	<code>{"UID":"1234","LEADTYPE":"GIVEN"}</code>

```
GET {base}?DATA=%7B%22UID%22%3A%221234%22%2C%22PID%22%3A%227%22%2C%22UNAME%22%3A%22vinod%22%7D&ACTION=ACHIVEDTARGET&ENCKEY=ABC123XYZ&SRC=KARMA&ISNEW=YES
```

⚠ **11 calls fire when this screen opens**, several in parallel. `LEADDASHBOARD` is called **four times** with overlapping parameters and `ACHIVEDTARGET` twice. Deduplicating would cut cold-start network traffic by roughly a third. See section 13, note 1.

`Application/Dashboard/Dashboard.dart` — *Dashboard* (older home screen)

Controller: `DashboardController` — shared, **no extra calls of its own**.

`Application/Dashboard/top_list.dart` — *TopList*

Controller: `TopListController` · **API calls: 1**

#	Action	What it does	DATA sent
1	<code>GETTOPCUSTOMERDETAILS</code>	The detail behind the "top customers" widget	<code>{"UID":"1234"}</code>

7.3 Contacts

`Application/Contacts/Contacts.dart` — *Contacts*

Controller: `ContactsController` · **API calls: 1**

#	Action	What it does	DATA sent	When
1	ANTRAUSER	Fetches the entire company staff directory	(empty)	on open · pull-to-refresh · retry

```
GET https://gateway.tallyhelp.com/crm/UpdateData.aspx?DATA=&ACTION=ANTRAUSER&ENCKEY=ABC123XYZ&SRC=KARMA&ISNEW=YES
```

Response: `{ "records": [ { "NAME", "DESIGNATION", "TEAMNAME", "teamOwner", ... } ] }`

Search does not call the server. It filters the cached copy — full walkthrough in section 3.

7.4 Data Point (customers)

"Data Point" is this app's word for a **customer**. **DPID** = customer id.

Application/DataPoint/DataPoints.dart — DataPoints (the customer list)

Controllers: `DataPointController`, `CreateDataPointController` · **API calls:** 1

#	Action	What it does	DATA sent
1	GETDATAPOINT	All customers assigned to this user	vinod

```
GET {base}?DATA=vinod&ACTION=GETDATAPOINT&ENCKEY=ABC123XYZ&SRC=KARMA&ISNEW=YES
```

Application/DataPoint/DataPointInfo.dart — DataPointInfo (customer 360 view)

Controller: `DataPointInfoController` · **API calls:** 5

#	Action	What it does	DATA sent
1	GETDPINF	The customer's master record	<code>{"DPID":"56789"}</code>
2	GETCALENTRYDATA	Their visit and call history	56789
3	BROCHUREMASTER	List of brochures available to send	(empty)
4	GETUSER	The contact people at this customer	56789
5	BROCHEREMAIL	submit — emails a chosen brochure to a contact	brochure id + email

Application/DataPoint/CreateDataPoint.dart — CreateDataPoint

Controller: `CreateDataPointController` · **API calls:** 3

#	Action	What it does	DATA sent
1	LEADSRC	Fills the "lead source" dropdown	(empty)
2	EPICMAST	Fills the "epicenter" dropdown	<code>{"UID":"1234"}</code>
3	NEWDATAPOINT	submit — creates the customer	URL-encoded JSON, below

```
{"MOBILE":"9876543210","EMAIL":"a@b.com","TALLYUSE":"Yes","URT":"1","WEBSITE":"","TALLYSRNO":"790012345","LOCATION":"Ahmedabad","PRODUCT":"Prime","EPICID":"12","LDSRC":"3"}
```

Application/DataPoint/ContactDetails.dart — ContactDetails

Controller: `ContactDetailsController` · **API calls:** 2

#	Action	What it does	DATA sent
1	GETUSER	Contact people at this customer	56789

#	Action	What it does	DATA sent
2	ADDCONTACT	submit — adds a new contact person	the new contact's fields

Application/DataPoint/ProductAndServices.dart — ProductAndServices

Controller: ServicesController · API calls: 1

#	Action	What it does	DATA sent
1	GETDPPRODUCTINFO	Which products/services this customer owns	{"DPID":"56789"}

Application/DataPoint/RecentActivity.dart — RecentActivity

Controller: RecentActivityController · API calls: 1

#	Action	What it does	DATA sent
1	OPPRECENTACTIVITY	Recent activity log for this customer	{"OPPID":"0","DPID":"56789"}

Application/DataPoint/TallySerial.dart — TallySerial

Controller: TallySerialController · API calls: 1 — GETCALLENTRYDATA , DATA 56789

Application/DataPoint/VisitHistory.dart — VisitHistory

Controller: VisitHistoryController · API calls: 1

#	Action	What it does	DATA sent
1	CALLENTRYDETAILS	Every visit this user made to this customer	vinod 56789 ← pipe format

```
GET {base}?DATA=vinod|56789&ACTION=CALLENTRYDETAILS&ENCKEY=ABC123XYZ&SRC=KARMA&ISNEW=YES
```

Application/DataPoint/DeliveryRequest.dart — DeliveryRequest

Controller: DeliveryRequestController · API calls: 1 — DELIVERYREQDATA , DATA {"UID":"1234"}

Application/DataPoint/Tickets.dart — Tickets

Controller: TicketController · API calls: 1 — uses the legacy Apis.sendData5

#	Action (inside data)	What it does	data sent
1	GETSUPPALLTICKET	Last 30 days of tickets for this customer	GETSUPPALLTICKET 30 56789

```
GET https://gateway.tallyhelp.com/crm/CustomerApp.aspx?data=GETSUPPALLTICKET|30|56789&ENCKEY=ABC123XYZ&SRC=KARMA
```

Application/DataPoint/CallBookingPage.dart — CallBookingPage

Controller: CallBookingController · API calls: 3

#	Action	What it does	DATA sent
1	GETCALLENTRYDATA	Existing call entries for this customer	56789
2	L1USER	Fills the "assign to L1 executive" dropdown	(empty)
3	CALLENTRY	submit — books the call	pipe-separated string

⚠ CALLENTRY is URL-encoded **only for L1 users**, sent raw for everyone else. See section 13, note 3.

AddressDetails.dart · DataPointDetails.dart · TallySerialNumber.dart

API calls: 0 — they display data handed to them by the previous screen.

7.5 Call Booking

Application/CallBooking/CallBooking.dart — CallBooking

Controller: `CallBookingController1` · API calls: 7

Actions here are declared as named constants at the top of the controller rather than inline strings — a good pattern the rest of the app doesn't follow yet.

#	Constant	Action	What it does	DATA sent
1	<code>callBookAction</code>	<code>CALLBOOK</code>	The list of booked calls	1234
2	<code>isWorkshopAction</code>	<code>ISWORKSHOP</code>	Is this call a workshop?	<code>{"DPID":"56789","CALLID":"9001"}</code>
3	<code>teamMemberAction</code>	<code>TEAMMEMBER</code>	Team list for the manager filter	1234
4	<code>deleteCallBookAction</code>	<code>DELETECALLBOOK</code>	submit — deletes a booking	<code>{"CALLID":"9001"}</code>
5	<code>callApproveAction</code>	<code>CALLAPPROVE</code>	submit — approves a call	approval JSON
6	<code>getCallDescAction</code>	<code>GETCALLEDISC</code>	Notes attached to a call	<code>{"CALLID":"9001"}</code>
7	<code>chkinoutDataAction</code>	<code>CHKINOUTDATA</code>	submit — records check-in / check-out	pipe-separated

Application/CallBooking/CallBookingL2.dart — CallBookingL2

Controllers: `CallBookingControllerL2`, `CallBookingController1` · API calls: 5

`CALLBOOK` · `TEAMMEMBER` · `CALLAPPROVE` · `GETCALLEDISC` · `CHKINOUTDATA` — the same set as above, for the support track.

Application/CallBooking/CallBookingDetails.dart — CallBookingDetails

Controller: `CallBookingDetailsController` · API calls: 3

#	Action	What it does	DATA sent
1	<code>CHKINOUTDATA</code>	submit — check in or out of a visit	pipe-separated
2	<code>CALLBOOK</code>	Reloads the booking list	1234
3	<code>ISWORKSHOP</code>	Whether to show the workshop form	<code>{"DPID":"56789","CALLID":"9001"}</code>

Application/CallBooking/AddSupportEntry.dart — AddSupportEntry

Controller: `AddSupportEntryController` (499 lines) · API calls: 5

#	Action	What it does	DATA sent
1	<code>CATEGORYTYPE</code>	Fills the support-category dropdown	vinod
2	<code>UPDATECALL</code>	submit (POST) — saves the support entry	form-encoded
3–5	<code>ChatGPT</code>	Rewrites the remark / observation / requirement text	free text

Calls 3–5 go to an **external LLM**, not the CRM server — `Apis.chatGptApi()`.

Application/CallBooking/ConveyanceEntry.dart — ConveyanceEntry (travel expenses)

Controller: `ConveyanceEntryController` · API calls: 7

#	Action	What it does	DATA sent
1	<code>GETSOURCE</code>	Fills the "travel source" dropdown	(empty)
2	<code>GETCONVAYANCE</code>	Existing expense entries	id
3	<code>CALLBOOK</code>	The visits an expense can attach to	vinod

#	Action	What it does	DATA sent
4	DELETECONVAYANCE	submit — deletes an entry	id
5	PASSCONV	submit — approves an expense	approval payload
6	PETROLRATE	Current petrol rate for the calculation	(empty)
7	SETCONVAYANCE	submit — saves the expense claim	expense payload

Note the spelling: the server uses **CONVAYANCE**, not "conveyance". Copy it exactly.

Application/CallBooking/PiwForm.dart — *PiwForm* (workshop feedback)

Controller: `PiwFormController` · API calls: 2

#	Action	What it does	DATA sent
1	WORKSHOPMST	The workshop question list	<code>{"CALLID":"9001"}</code>
2	SAVEWORKSHOPPREF	submit (POST) — saves the answers	<code>{"DATA":[...answers]}</code>

Application/CallBooking/SupportList.dart — *SupportList*

Controller: `SupportListController` · API calls: 1 — `EXISTSUPENTRY`, DATA `9001`

Application/CallBooking/SupportDetails.dart — *SupportDetails*

Controller: `SupportDetailsController` · API calls: 1 — `SUPENTRY`, DATA `9001`

Application/CallBooking/CreateLead.dart — *CreateLead*

Controller: `CreateLeadController` · API calls: 4

#	Action	What it does	DATA sent
1	LEADSRC	Lead-source dropdown	(empty)
2	LEADSRC	The same call again in <code>getLeadData()</code>	(empty)
3	GETTALLYSRLOC	Looks up a location from a Tally serial number	<code>{"TALLYSRNO":"790012345"}</code>
4	NEWLEAD	submit — creates the lead	lead payload

ColdCalling.dart · NearByCustomer.dart

API calls: 0 — local and GPS-driven UI.

7.6 Lead

Application/Lead/Lead.dart — *Lead*

Controller: `LeadController` · API calls: 1

#	Action	What it does	DATA sent
1	LEADDASHBOARD	Leads, filtered by type	<code>{"LEADTYPE":"GIVEN","UID":"1234"}</code>

`LEADTYPE` is the on-screen filter uppercased — `GIVEN` or `RECEIVED`. The default differs by role (see the role doc).

Application/Lead/LeadReceived.dart — *LeadReceived*

Controller: `LeadReceivedController` · API calls: 3

#	Action	What it does	DATA sent
1	LEADLIST or RCVLEADLIST	Leads given, or leads received — the action itself switches with the tab	{"USERID":"1234","LEADTYPE":"Given"}
2	TCKLEADSTYPE	Lead-type dropdown	(empty)
3	LEADTICKET	submit — turns a lead into a ticket	{"LEADID":"77","LEADTYPE":"2","DESCR":"..."}

Application/Lead/CreateLead1.dart — CreateLead1

Controller: CreateLeadController1 · API calls: 2 — LEADSRC , then NEWLEAD (submit)

Application/Lead/UpdateLead.dart — UpdateLead

Controller: UpdateLeadController · API calls: 2

#	Action	What it does	DATA sent
1	GIVENLEAD	Loads one lead's details	77
2	UPDATELEAD	submit — updates its status	77~2~Followed up~1234~56789

```
GET {base}?DATA=77~2~Followed up~1234~56789&ACTION=UPDATELEAD&ENCKEY=ABC123XYZ&SRC=KARMA&ISNEW=YES
      |      |      |      |      |
      leadId status comment   userId DPID
```

This is the **only** call in the entire app using ~ as a separator. Do not copy it.

7.7 Opportunity

Application/Opportunity/OpportunityDetails.dart — OpportunityDetails

Controller: OpportunityController · API calls: 1

#	Action	What it does	DATA sent
1	GETOPP	Opportunities — for the whole user, or for one customer	{"UID":"1234","DPID":""} or {"UID":"","DPID":"56789"}

Which one depends on how the screen was opened: from the drawer → UID ; from a customer → DPID .

Application/Opportunity/OpportunityData.dart — OpportunityData

Controller: OpportunityDataController · API calls: 3

#	Action	What it does	DATA sent
1	OPPRECENTACTIVITY	Activity log for this opportunity	{"OPPID":"4501","DPID":"56789"}
2	PROPOSALLIST	Proposals sent	{"OPPID":"4501","DPID":"56789"}
3	PREFORMAINVOIC	Proforma invoice	{"OPPID":"4501","DPID":"56789","ISMAIL":"220"}

Application/Opportunity/Status.dart — Status

Controller: StatusController · API calls: 2

#	Action	What it does	DATA sent
1	CLOSEREASONMAST	Dropdown of reasons for closing	(empty)
2	OPPSTATUSUPDATE	submit — changes the status	JSON below

```
{"DPID":"56789","OPPID":"4501","STATUS":"Closed","REASONID":"3",
"REMARK":"Budget not approved","POSDATE":""}
```

Application/Opportunity/UpdateOpportunity.dart — UpdateOpportunity

Controller: UpdateOpportunityController · **API calls:** 2 — PROPOSALSOURCE ({"DPID":"56789"}) and GETCALLENTTRYDATA (56789)

7.8 Ticket

Application/Ticket/Ticket.dart — Ticket

Controller: TicketListController (in ticketController1.dart) · **API calls:** 1

#	Action	What it does	DATA sent
1	GETANTRATICKET	The user's ticket list	{"uid":"1234","status":"..."}

Application/Ticket/CreateTicket.dart — CreateTicket

Controller: CreateTicketController · **API calls:** 2

#	Action	What it does	DATA sent
1	GETTCKMAST	Category and subject dropdowns	(empty)
2	ANTRATICKET	submit (POST) — raises the ticket	JSON below

```
{"categoryid":"12","descr":"Printer not working","uid":"1234","priority":"High","duplicate":"0"}
```

Note: this payload uses **lowercase** keys, unlike most of the app.

Application/Ticket/RemarkList.dart — RemarkList

Controller: RemarkListController · **API calls:** 1 — ANTRATICKETREMARK , DATA {"tckid":"TCK00123"}

7.9 Ticket Status

Application/TicketStatus/TicketStatus.dart — TicketStatus

Controller: TicketStatusController · **API calls:** 2

#	Action	What it does	DATA sent
1	GETSUPPALLPENDTICKET	Pending tickets, last 30 days	30 1234 ← days userId
2	ACTTCKUPDATE	submit — updates a ticket	TCK00123 1234

Application/TicketStatus/TickerDetails.dart — TicketDetails

Controller: TicketDetailsController1 · **API calls:** 1 — legacy sendData5

#	Action (inside data)	What it does	data sent
1	TICKETDETAILS	One ticket with its full interaction history	TICKETDETAILS TCK00123

```
GET https://gateway.tallyhelp.com/crm/CustomerApp.aspx?data=TICKETDETAILS|TCK00123|&ENCKEY=ABC123XYZ&SRC=KARMA
```

Response: ROOT.ticket_details , with ROOT.ticket_details.interaction[] inside it.

Application/TicketStatus/TicketStatusDetails.dart — TicketStatusDetails

Controller: TicketDetailsController · API calls: 1 — same TICKETDETAILS call

⚠ Two controllers do the same thing. Note the typo in one filename: tickerDetailsController1.dart — "ticker", not "ticket". See section 13, note 8.

7.10 Support & customer calls

Application/Support/Support.dart — Support

Controller: SupportController · API calls: 1 — AVAILABLEUSER, DATA empty (which support staff are free right now)

Application/Customer/MyCustomerCall.dart — MyCustomerCall

Controllers: MyCustomerCallController, DashboardController · API calls: 2

#	Action	What it does	DATA sent
1	MYCALLDTLS	This user's customer calls	{"UID":"1234"}
2	SUPCALLCANCEL	submit — cancels a call with a reason	{"CALLID":"9001","REMARK":"Customer unavailable"}

7.11 Reports ("Operations" in the drawer)

Application/Report/Reports.dart — Reports

API calls: 0 — a menu. (Which cards appear depends on role — see the role doc.)

Application/Report/LeaveReport.dart — LeaveReport

Controller: LeaveReportController · API calls: 3

#	Action	What it does	DATA sent
1	ATTENDANCE	Monthly attendance record	EMP0042 Sep 2026
2	ANTRAHOLIDAY	Company holiday list	(empty)
3	CREATELEAVE	submit — applies for leave	leave JSON

Application/Report/HolidaysList.dart — HolidaysList

Controller: LeaveReportController (shared) · API calls: 1 — ANTRAHOLIDAY

Application/Report/LeaveStatus.dart — LeaveStatus

Controller: LeaveStatusController · API calls: 5

#	Action	What it does	DATA sent
1	TEAMMEMBER	Team dropdown (managers only)	1234
2	GETLEAVEDATA	Leave records	{"UID","MONTH","STATUS"}
3	DELETELEAVE	submit — cancels a request	{"LEAVEID":"301"}
4	UPDATELEAVE	submit — edits a request	URL-encoded JSON
5	APPROVELEAVE	submit — approves or rejects	JSON below

```
{"LEAVEID":"301","UID":"1234","UNAME":"vinod","STATUS":"Approved","REASON":"OK"}
```

⚠️ Approve/reject is hidden in the UI for non-managers, but **the request carries no role field** — the server must enforce it. See the role doc, risk 4.

Application/Report/ConveyReport.dart — ConveyReport

Controller: ConveyReportController · API calls: 2

#	Action	What it does	DATA sent
1	GETUSERDETAILS	User details (<i>note the typo — server-side</i>)	vinod
2	CONVREPORT	Travel expense report for a month	{"UNAME":"vinod","FROMDATE":"1 Sep 2026"}

Application/Report/TargetReport.dart — TargetReport

Controller: TargetReportController · API calls: 2

#	Action	What it does	DATA sent
1	TEAMMEMBER	Team dropdown	1234
2	whatever Get.arguments['action'] holds	The report itself	{"UID":"1234","MONTH":9}

⚠️ The action is passed in from the previous screen, so this one file serves several report types. You cannot tell which endpoint it hits by reading this file alone — you must find its callers.

Application/Report/MonthlyReport.dart — API calls: 0 (renders passed-in data)

7.12 Outstanding

Application/Outstanding/Outstanding.dart — Outstanding

Controller: OutstandingController · API calls: 1

#	Action	What it does	DATA sent
1	USEROUTSTANDING	Unpaid customer invoices	{"UNAME":"vinod","UID":"1234","UST":"USER"}

7.13 Epicenters

Application/Epicenters/EpicentersDetails.dart — EpicenterDetails

Controller: EpicentersController · API calls: 1 — GETEPCDATA, DATA {"UID":"1234","UNAME":"vinod"}

Application/Epicenters/EpicentersPage2.dart — EpicentersPage2

Controller: EpicentersPage2Controller · API calls: 0

Receives its list through Get.arguments['data'] and filters locally. A good example of a screen that needs no network call at all.

EpicentersDetails2.dart · epicenter/epicenter_screen.dart — API calls: 0

7.14 Escalation

Application/Escalation/EscalationChooseType.dart — API calls: 0 (a picker)

Application/Escalation/EscalationInternalForm.dart — EscalationInternalForm

Controller: EscalationController · API calls: 2

#	Action	What it does	DATA sent
1	ESCALATIONMST	Categories, improvement areas, users	<code>{"uid":"1234"}</code>
2	SAVEESCALATION	submit (POST)	JSON below

```
{"userid":"1234","dpid":"0","exeid":"0","catid":"5","imprid":"2","touserid":"88",
"subcatid":"0","background":"...","observation":"...","Recommendation":"..."}
```

Application/Escalation/EscalationTicketForm.dart — EscalationTicketForm

Controller: `EscalationController` · API calls: 3

ESCALATIONMST · DPTICKETINFO (`{"dpid":"56789"}` — the customer's tickets) · SAVEESCALATION (adds `"ticket"`, a real `"dpid"` and `"exeid"`)

Application/Escalation/EscalationSuccess.dart — API calls: 0

7.15 AGH — Antra Golden Hour

Application/AGH/AGHDashboard.dart — AGHDashboard

Controller: `AGHController` (1173 lines) · API calls: 4 — all on the **legacy** `Apis` layer

#	Action	What it does	DATA sent	Helper
1	GETAGH	Sessions for this user	<code>{"captain_id":"1234"}</code>	<code>sendData4</code>
2	GETTOPIC	Topic list	<code>''</code>	<code>sendData4</code>
3	AGHSCHEDULE	submit — schedules a session	payload map	<code>sendDataPost</code>
4	SAVEAGH	submit — saves the outcome	session map incl. <code>'role'</code>	<code>sendDataPost</code>

```
GET {base}?DATA=&ACTION=GETTOPIC&ISNEW=YES&ENCKEY=ABC123XYZ&SRC=KARMA
```

⚠️ `"&ISNEW=YES"` is glued onto the ACTION value

Two quirks here. First, `&ISNEW=YES` is smuggled into the `ACTION` string instead of being its own parameter. Second, the field is called `captain_id` but always carries the current user's id — **even when that user is not a captain**.

`SAVEAGH` is also the **only call in the entire app that tells the server the user's role**.

The other 6 AGH screens

`AGHCheckIn` · `AGHCompleted` · `AGHMemberLearn` · `AGHNotifications` · `AGHOutcome` · `AGHScheduleSession` — all share `AGHController`. **No extra endpoints**; they read its cached state and trigger `AGHSCHEDULE` / `SAVEAGH`.

7.16 ANS

Application/ANS/ans_screen.dart — AnsScreen

Controller: `AnsController` · API calls: 2

#	Action	What it does	DATA sent
1	GETANS	The ANS entry list	<code>{"login_id":"1234"}</code>
2	ANSUSER	User dropdown	<code>{"login_id":"1234"}</code>

Application/ANS/create_ans_screen.dart — CreateAnsScreen

Controller: `CreateAnsController` · API calls: 2 — `ANSUSER`, then `ANS` (submit)

7.17 PBL Report

Application/PblReport/PblReport.dart — PblReport

Controller: `PblController` · API calls: 2 — `GETPBL` and `ANSUSER`, both `{"login_id":"1234"}`

Application/PblReport/create_pbl_screen.dart — CreatePblScreen

Controller: `CreatePblController` · API calls: 2 — `ANSUSER`, then `ANS` (submit)

PBL reuses the ANS endpoints (`ANSUSER`, `ANS`). They are the same backend feature.

7.18 Products

Application/Products/Products.dart — Products

Controller: `ProductController` · API calls: 4 — all legacy, all on `customerBaseUrl`

#	Action	What it does	DATA sent	Helper
1	<code>GETMODULELIST</code>	Tally module catalogue	<code>""</code>	<code>sendData1</code>
2	<code>GETAPPLIST</code>	App catalogue	<code>""</code>	<code>sendData1</code>
3	<code>GETBOOSTERCAT</code>	Booster categories	<code>""</code>	<code>sendData1</code>
4	<code>GETBOOSTERLIST { id}</code>	Boosters in a category	<i>(action only)</i>	<code>sendData2</code>

GET <https://gateway.tallyhelp.com/crm/CustomApp.aspx?DATA=&ACTION=GETMODULELIST&ENCKEY=ABC123XYZ&SRC=KARMA>
GET <https://gateway.tallyhelp.com/crm/CustomApp.aspx?ACTION=GETBOOSTERLIST|{|id}&ENCKEY=ABC123XYZ&SRC=KARMA>

Application/Products/ProductDetails.dart — API calls: 0

7.19 Profile

Application/Profile/profile.dart — API calls: 0 — reads `DataInfo` globals

Application/Profile/EditProfile.dart — EditProfile

Controller: `ProfileController` · API calls: 2

#	Action	What it does	DATA sent
1	<code>UPDATEUSERDETAILS</code>	submit — saves the profile	5 fields, pipe-separated
2	<i>(legacy <code>Apis.sendData</code>)</i>	Profile image follow-up	—

GET [{base}?DATA=Vinod Sharma|Sr. Executive|9876543210|vinod@antraweb.co.in|vinod&ACTION=UPDATEUSERDETAILS&...](#)
 fullName designation mobile email username

7.20 Notifications

`Application/Notification/NotificationScreen.dart` — *NotificationScreen*

Controller: `NotificationListController` · **CRM API calls:** 0

Driven by Firebase Cloud Messaging plus a raw `http.get` in `notificationService.dart`. Registered as a **permanent** controller in `InitialBindings`, so it stays alive for the whole session.

7.21 Screens with no API calls

Screen	File
Company	<code>Application/Company/Company.dart</code>
Company Details	<code>Application/Company/CompanyDetails.dart</code>
Data Scope	<code>Application/data_scope/data_scope.dart</code>
Drawer	<code>Application/Drawer/DrawerWidget.dart</code> — navigation + logout only

8. Dictionary of all 88 actions

Alphabetical. Use this when you see an `ACTION` in a log and don't know what it is.

8.1 The 75 declared as plain strings

ACTION	What it does	Declared at
<code>ACHIVETARGET</code>	Sales target achieved so far (<i>note the spelling — one "E"</i>)	<code>dashboardController.dart:148</code>
<code>ACTTCKUPDATE</code>	Update a ticket's action/status	<code>ticketStatusController.dart:88</code>
<code>ADDCONTACT</code>	Add a contact person to a customer	<code>ContactDetailsController.dart:79</code>
<code>ANS</code>	Submit an ANS / PBL entry	<code>create_ans_controller.dart:133</code>
<code>ANSUSER</code>	User dropdown for ANS and PBL	<code>create_ans_controller.dart:76</code>
<code>ANTRAHOLIDAY</code>	Company holiday list	<code>leaveReportController.dart:102</code>
<code>ANTRATICKET</code>	Create a support ticket	<code>createTicketController.dart:130</code>
<code>ANTRATICKETREMARK</code>	Remarks/comments on a ticket	<code>remark_list_controller.dart:31</code>
<code>ANTRAUSER</code>	Full company staff directory	<code>contactsController.dart:30</code>
<code>APPROVELEAVE</code>	Approve or reject a leave request	<code>leaveStatusController.dart:269</code>
<code>APPVERSION</code>	Latest app version, for the update prompt	<code>authController.dart:267</code>
<code>ATTENDANCE</code>	Monthly attendance record	<code>leaveReportController.dart:74</code>
<code>AVAILABLEUSER</code>	Which support staff are free right now	<code>supportController.dart:29</code>
<code>BROCHEREMAIL</code>	Email a brochure to a contact (<i>note the spelling</i>)	<code>DataPointInfoController.dart:135</code>
<code>BROCHUREMASTER</code>	List of available brochures	<code>DataPointInfoController.dart:85</code>
<code>CALLAPPROVE</code>	Approve a booked call (<i>one "P"</i>)	<code>callBookingControllerL2.dart:108</code>
<code>CALLBOOK</code>	List of booked calls	<code>callBookingDetailsController.dart:125</code>
<code>CALLENTRY</code>	Submit a call booking	<code>callBookingController.dart:454</code>
<code>CALLENTRYDETAILS</code>	Visit history for one customer	<code>visitHistoryController.dart:21</code>
<code>CHKINOUTDATA</code>	Record a visit check-in / check-out	<code>callBookingDetailsController.dart:62</code>

ACTION	What it does	Declared at
CLOSEREASONMAST	Dropdown: reasons for closing an opportunity	statusController.dart:40
CONVREPORT	Travel expense report	conveyReportController.dart:32
CREATELEAVE	Apply for leave	leaveReportController.dart:180
DELETELEAVE	Cancel a leave request	leaveStatusController.dart:104
DELIVERYREQDATA	Delivery requests for a customer	deliveryRequestController.dart:24
DPTICKETINFO	A customer's tickets, for escalation	escalationController.dart:188
EPICMAST	Dropdown: epicenters	createDataPointController.dart:116
ESCALATIONMST	Escalation master data (categories, users)	escalationController.dart:69
EXISTSUPENTRY	Existing support entries for a call	supportListController.dart:26
GETANTRATICKET	Ticket list	ticketController1.dart:63
GETCALLEDSC	Notes attached to a call (<i>note the spelling</i>)	callBookingControllerL2.dart:134
GETCALLENTYDATA	Call/visit entries for a customer	callBookingController.dart:115
GETDATAPOINT	The user's customer list	dataPointController.dart:50
GETDPINF	One customer's master record	DataPointInfoController.dart:44
GETDPPRODUCTINFO	Products a customer owns	ServicesController.dart:23
GETEPCDATA	Epicenter data	epicentersController.dart:32
GETLEAVEDATA	Leave records	leaveStatusController.dart:90
GETOPP	Opportunity list	dashboardController.dart:602
GETPBL	PBL report data	pbl_controller.dart:37
GETSUPPALLPENDTICKET	Pending support tickets	ticketStatusController.dart:68
GETSUPRANK	Support ranking	dashboardController.dart:712
GETTALLYSRLOC	Location from a Tally serial number	createLeadController.dart:189
GETTCKMAST	Dropdowns: ticket categories and subjects	createTicketController.dart:59
GETTOPCUSTOMER	Top customers widget	dashboardController.dart:633
GETTOPCUSTOMERDETAILS	Detail behind the top-customers widget	top_list_controller.dart:76
GETUSER	Contact people at a customer	ContactDetailsController.dart:43
GETUSERDITAILS	User details (<i>typo is server-side — keep it</i>)	conveyReportController.dart:25
GIVENLEAD	One lead's details	updateLeadController.dart:37
ISWORKSHOP	Is this call a workshop?	callBookingDetailsController.dart:164
L1USER	Dropdown: L1 executives	callBookingController.dart:184
LEADDASHBOARD	Lead counts and list	leadController.dart:31
LEADSRC	Dropdown: lead sources	createLeadController1.dart:32
LEADTICKET	Turn a lead into a ticket	leadReceivedController.dart:98
LOGIN	Sign in — populates every global	authController.dart:111
MYCALLDTLS	The user's customer calls	myCustomerCallController.dart:37
NEWDATAPOINT	Create a customer	createDataPointController.dart:257
NEWLEAD	Create a lead	createLeadController1.dart:114
OPPRECENTACTIVITY	Recent activity on an opportunity/customer	OpportunityDataController.dart:36
OPPSTATUSUPDATE	Change an opportunity's status	statusController.dart:90

ACTION	What it does	Declared at
PREFORMAINVOIC	Proforma invoice (<i>note: truncated spelling</i>)	OpportunityDataController.dart:89
PROPOSALLIST	Proposals for an opportunity	OpportunityDataController.dart:61
PROPOSALSOURCE	Dropdown: proposal sources	updateOpportunityController.dart:65
REGISTERID	Register the device + push token	authController.dart:193
SAVEESCALATION	Submit an escalation	escalationController.dart:291
SAVEWORKSHOPREF	Submit workshop feedback	PiwFormController.dart:104
SUPCALLCANCEL	Cancel a support call	myCustomerCallController.dart:62
SUPDASHBOARD	Support dashboard counts	dashboardController.dart:387
SUPENTRY	One support entry's detail	SupportDetailsController.dart:18
TCKLEADSTYPE	Dropdown: lead types for tickets	leadReceivedController.dart:70
TEAMMEMBER	Team list — the manager's dropdown	targetReportController.dart:65
UPDATELEAD	Update a lead's status	updateLeadController.dart:76
UPDATELEAVE	Edit a leave request	leaveStatusController.dart:199
UPDATEUSERDETAILS	Save the user's own profile	profileController.dart:25
USEROUTSTANDING	Unpaid customer invoices	outstandingController.dart:25
WORKSHOPMST	Workshop question list (PIW form)	PiwFormController.dart:41

8.2 The 13 declared as named constants

Better practice — the value lives in one place. Only four controllers do this.

Constant	Value	What it does	File
actionGetSource	GETSOURCE	Dropdown: travel sources	conveyanceEntryController.dart:48
actionGetConveyance	GETCONVAYANCE	Existing expense entries	:49
actionDeleteConveyance	DELETECONVAYANCE	Delete an expense entry	:50
actionPassConv	PASSCONV	Approve an expense	:51
actionCallBook	CALLBOOK	Visits an expense can attach to	:52
actionPetrolRate	PETROLRATE	Current petrol rate	:53
actionSetConveyance	SETCONVAYANCE	Save the expense claim	:54
updateCallAction	UPDATECALL	Save a support entry	AddSupportEntryController.dart:12
categoryTypeAction	CATEGORYTYPE	Dropdown: support categories	:13
getAnsAction	GETANS	ANS entry list	ans_controller.dart:26
deleteCallBookAction	DELETECALLBOOK	Delete a call booking	callBookingController1.dart:14

Constant	Value	What it does	File
(also) <code>ansUserAction</code> , <code>callBookAction</code> , <code>teamMemberAction</code> , <code>callApproveAction</code> , <code>isWorkshopAction</code> , <code>getCallDescAction</code> , <code>chkinoutDataAction</code>	—	duplicates of actions already listed above	—

8.3 Actions with no `ACTION=` parameter

These hide the action inside `data` . Easy to miss when searching the codebase.

Action	Sent as	File
<code>GETSUPPALLTICKET</code>	<code>data=GETSUPPALLTICKET 30 {dpId}</code>	<code>ticketController.dart:31</code>
<code>TICKETDETAILS</code>	<code>data=TICKETDETAILS {ticketNo} </code>	<code>tickerDetailsController1.dart:21</code>
<code>TICKETDETAILS</code>	<code>data=TICKETDETAILS {ticket} </code>	<code>ticketDetailsController.dart:23</code>

8.4 Legacy-layer actions

Action	What it does	Helper	Endpoint
<code>GETAGH</code>	AGH sessions	<code>sendData4</code>	<code>baseUrl</code>
<code>GETTOPIC</code>	AGH topics	<code>sendData4</code>	<code>baseUrl</code>
<code>AGHSCHEDULE</code>	Schedule an AGH session	<code>sendDataPost</code>	<code>baseUrl</code>
<code>SAVEAGH</code>	Save an AGH outcome	<code>sendDataPost</code>	<code>baseUrl</code>
<code>GETMODULELIST</code>	Tally module catalogue	<code>sendData1</code>	<code>customerBaseUrl</code>
<code>GETAPPLIST</code>	App catalogue	<code>sendData1</code>	<code>customerBaseUrl</code>
<code>GETBOOSTERCAT</code>	Booster categories	<code>sendData1</code>	<code>customerBaseUrl</code>
<code>GETBOOSTERLIST</code>	Boosters in a category	<code>sendData2</code>	<code>customerBaseUrl</code>

8.5 Actions that change at runtime

Two places pick the action **while the app is running**, so you cannot find them by grepping for a name:

Where	How it decides
<code>leadReceivedController.dart:35</code>	<code>lead == "Given" ? "LEADLIST" : "RCVLEADLIST"</code>
<code>targetReportController.dart:50</code>	<code>action: args['action']</code> — passed in from the previous screen

9. The 6 payload formats

Before editing any call, check which format it uses.

Format	Looks like	Used by
Empty	<code>DATA=</code>	<code>ANTRAUSER</code> , <code>LEADSRC</code> , <code>BROCHUREMASTER</code> , <code>CLOSEREASONMAST</code> , <code>AVAILABLEUSER</code>
Plain value	<code>DATA=vinod</code> · <code>DATA=56789</code>	<code>GETDATAPOINT</code> , <code>CALLBOOK</code> , <code>TEAMMEMBER</code> , <code>GETUSER</code> , <code>SUPENTRY</code>
Pipe	<code>DATA=vinod 56789</code>	<code>CALLENTRYDETAILS</code> , <code>ATTENDANCE</code> , <code>ACTTCKUPDATE</code> , <code>UPDATEUSERDETAILS</code> , <code>GETSUPPALLPENDTICKET</code>
Tilde ~	<code>DATA=77~2~text~1234~56789</code>	<code>UPDATELEAD</code> — the only one
JSON	<code>DATA={"UID":"1234"}</code>	most modern actions

Format	Looks like	Used by
URL-encoded JSON	DATA=%7B%22UID%22...	LOGIN , NEWDATAPOINT , UPDATELEAVE , CALLENTRY (L1 only)

The trap

JSON is inconsistently encoded. Some calls wrap with `Uri.encodeComponent` , most don't. Un-encoded JSON in a GET query string breaks the moment the payload contains `&` , `#` or `+` — the server sees a truncated request and the screen silently shows nothing.

Worst example — `callBookingController.dart:451` :

```
data: DataInfo.desCat.value == "L1"
    ? Uri.encodeComponent(callBookingData) // L1 users → encoded
    : callBookingData,                    // everyone else → raw
```

The same endpoint gets differently encoded payloads **depending on who is logged in**.

Rule going forward: always `Uri.encodeComponent` any JSON payload. It is safe even when unnecessary.

10. Session globals — `lib/Constants/dataInfo.dart`

Nearly every payload is built from these. They are set **once at login** and read directly by every controller.

Global	Appears in DATA as	Demo	Set at
<code>DataInfo.encKey</code>	ENCKEY on every call	ABC123XYZ	login
<code>DataInfo.userId</code>	UID , USERID , uid , login_id , captain_id	1234	login
<code>DataInfo.username</code>	UNAME , USER	vinod	login
<code>DataInfo.pid</code>	PID	7	login
<code>DataInfo.enrollId</code>	ENROLLID	EMP0042	login
<code>DataInfo.dpId</code>	DPID	56789	on selecting a customer
<code>DataInfo.desCat</code>	branching only	L1	login
<code>DataInfo.rollId</code>	branching only	1	login
<code>DataInfo.tcId</code>	role in SAVEAGH	7	login
<code>DataInfo.titleId</code>	branching only	24	login
<code>DataInfo.appVersion</code>	APPVERSION payload	10.0.0	app start

⚠ These are mutable globals, not read-only session data. When a manager selects a team member on the dashboard, `dashboardController.dart:110-111` **overwrites** `enrollId` and `desCat` . Every request made afterwards carries the *team member's* values. Full explanation in the role doc, risk 1.

Also worth knowing: `dataInfo.dart:51` holds a **hardcoded Google Maps API key** with a comment saying it must be rotated. That is still outstanding.

11. How to add a new screen

The complete recipe. Follow it and your screen will match the rest of the app.

1 — Create the controller · `lib/Controller/myThingController.dart`

```
import 'package:karma/Constants/Library.dart';

class MyThingController extends GetxController {
  RxList<dynamic> items      = [].obs; // the data
  RxBool             isLoading = false.obs; // spinner?
  RxBool             hasError  = false.obs; // error state?

  @override
  void onInit() {
    getData(); // fires as soon as the controller is created
    super.onInit();
  }

  Future<void> onRefresh() async => getData(); // for pull-to-refresh

  Future<void> getData() async {
    isLoading.value = true;
    hasError.value  = false;
    try {
      final value = await Api().fetchApi(
        data: json.encode({'UID': DataInfo.userId.value}),
        action: "MYACTION",
      );
      if (value != null && value.success) {
        items.value = value.data['records'] ?? [];
      } else {
        hasError.value = true;
      }
    } catch (e) {
      hasError.value = true;
      if (kDebugMode) print("MyThing error: $e");
    } finally {
      isLoading.value = false; // ALWAYS in finally
    }
  }
}
```

2 — Create the screen · `lib/Application/MyThing/MyThing.dart`

```
import 'package:karma/Constants/Library.dart';
import 'package:karma/Widgets/AsyncStateView.dart';

class MyThing extends GetView<MyThingController> {
  const MyThing({super.key});

  @override
  Widget build(BuildContext context) {
    Get.put(MyThingController());
    return Scaffold(
      appBar: AppBarWidget(title: "My Thing"),
      body: Obx(() => RefreshIndicator(
        onRefresh: controller.onRefresh,
        child: AsyncStateView(
          isLoading: controller.isLoading.value,
          hasError: controller.hasError.value,
          isEmpty: controller.items.isEmpty,
          onRetry: controller.getData,
          child: ListView.builder(
            physics: const AlwaysScrollableScrollPhysics(), // needed for pull-to-refresh
            itemCount: controller.items.length,
            itemBuilder: (context, i) => tileWidget(controller.items[i]),
          ),
        ),
      )),
    );
  }

  Widget tileWidget(var data) => Card(child: ListTile(title: Text(data['NAME'])));
}
```

3 — Export both from the barrel · `lib/Constants/Library.dart`

```
export 'package:karma/Controller/myThingController.dart';
export 'package:karma/Application/MyThing/MyThing.dart';
```

Skip this and other files won't see your classes.

4 — Add it to the drawer, if it needs a menu entry · `DrawerWidget.dart`

```
tileWidget(
  onTap: () { Get.back(); Get.to(() => const MyThing()); },
  title: "My Thing",
  iconName: notesIcon,
),
```

Role-restricted? Wrap it — and document it in `ROLE-ACCESS-DOCUMENTATION.md` :

```
DataInfo.desCat.value == "L1"
  ? tileWidget(...)
  : const SizedBox(),
```

5 — Update both docs

Add your screen to section 7 and your action to section 8 here, and to the role doc if it has any role gating.

Checklist

- [] Controller in `lib/Controller/` , screen in `lib/Application/<Feature>/`
- [] Screen uses `GetView<YourController>` — not `Get.find` scattered about
- [] `isLoading` set in `finally` , never only on the happy path
- [] `hasError` set, and `AsyncStateView` used — don't hand-roll the states
- [] Any JSON payload wrapped in `Uri.encodeComponent`
- [] No `Api` import in the screen file
- [] No widget code in the controller
- [] Both files exported from `Library.dart`
- [] `Api` , not `Apis`

12. Glossary

Term	Meaning
ACTION	The query parameter naming what you want from the server. The app has 88.
DATA	The query parameter carrying your input. Six different formats — section 9.
ENCKEY	Session key from <code>LOGIN</code> , sent on every call. Proves who you are.
Data Point / DPID	This app's word for a customer and its id.
Barrel file	<code>Library.dart</code> — one file re-exporting 158 others, so screens need one import.
Controller	The GetX class holding a screen's data and logic.
<code>GetView<T></code>	A screen base class that provides a free <code>controller</code> property of type <code>T</code> .
<code>Get.put()</code>	Creates a controller. This is what triggers the first API call , via <code>onInit</code> .
<code>onInit()</code>	GetX lifecycle hook, called automatically when a controller is created.
<code>Rx</code> / <code>.obs</code>	An observable value. Assigning to it redraws any <code>Obx</code> watching it.

Term	Meaning
<code>Obx</code>	A widget that rebuilds automatically when an <code>Rx</code> inside it changes.
<code>AsyncStateView</code>	Shared widget choosing between spinner / error / empty / content.
<code>DataInfo</code>	Global session values — <code>userId</code> , <code>encKey</code> , role flags. Section 10.
<code>L1</code>	The sales/field track (<code>desCat == "L1"</code>). See the role doc.
<code>AGH</code>	"Antra Golden Hour" — a coaching-session module with Captain and Member roles.
<code>PIW</code>	The workshop feedback form (<code>WORKSHOPMST</code> / <code>SAVEWORKSHOPREF</code>).
<code>PBL / ANS</code>	Two report modules that share the same backend endpoints.
<code>submit</code>	In this doc: a call that <i>writes</i> to the server, rather than reading.

13. Engineering notes

Things worth fixing, most impactful first. Each is a real observation from the source, not a style preference.

1 — The dashboard fires 11 calls on open. `LEADDASHBOARD` is called four times and `ACHIVEDTARGET` twice, on overlapping parameters. Deduplicating would cut cold-start network traffic by roughly a third. *Biggest single win available.*

2 — Two API layers coexist. `Api` gives you timeouts, the slow-network warning, debug logging and a response wrapper. `Apis` gives you none of those. AGH, Products, Tickets and Profile are still on the old one, so those screens can hang for 50 seconds with no feedback. **Migrating them is the highest- value cleanup in the codebase.**

3 — JSON encoding is inconsistent, and in one case role-dependent. See section 9. `CALLENTRY` encodes for L1 and sends raw for everyone else, so the same endpoint receives two different formats depending on who is logged in.

4 — `Api()` is constructed on every single call. `Api().fetchApi(...)` builds a new `Dio` instance and a fresh interceptor stack each time. A single shared instance would be cheaper and would allow connection reuse.

5 — `_fetchApi` calls through a second `Api()`. `Api.dart:64` reads `Api().sendRequest` instead of the instance's own `_dio` — which means the `baseUrl` constructor argument is dead code and never has any effect.

6 — Users can see two error messages for one failure. `Api.fetchApi` shows a snackbar on failure *and* returns the failed response; several controllers then show their own. Let `Api` own the snackbar; controllers should only set `hasError`.

7 — Only 20 of roughly 50 list screens use `AsyncStateView`. The rest hand-roll loading and empty states, inconsistently. `Contacts.dart` is the cleanest reference — copy that shape.

8 — Duplicate and near-duplicate controllers. `callBookingController.dart` / `callBookingController1.dart` · `createLeadController.dart` / `createLeadController1.dart` · `ticketDetailsController.dart` / `tickerDetailsController1.dart` (note the typo: "ticker"). Confirm which is live in each pair and delete the other.

9 — `Get.put()` inside `build()`. Used in 58 screens. It works because `GetX` returns the existing instance, but route `Bindings` is the intended pattern and avoids surprises around `Get.off` / `Get.back` lifecycles. `InitialBindings` already exists — it just isn't used per route.

10 — No typed models. `lib/Models/` holds only `DeviceInfo` and `UserInfo`. Every API response is passed around as `dynamic` / `Map`, so field access like `element['teamOwner']` fails **silently** when the backend renames a column. Typed models on the top ten responses would surface these at compile time.

11 — Naming is inconsistent — `contactsController.dart` (camel) vs `ContactDetailsController.dart` (Pascal) vs `pbl_controller.dart` (snake). Pick one before the file count grows further.

12 — `lib/core/` , `lib/domain/` , `lib/test/` are empty. Delete them, or adopt them deliberately.

13 — The production URL is hardcoded and the dev server is a commented-out line. A `--dart-define` flavour would remove the risk of shipping a local URL by accident.

14 — A Google Maps API key is committed in `dataInfo.dart:51`, with a comment saying it needs rotating in GCP. Still outstanding.

End of document. For who-can-see-what, see [ROLE-ACCESS-DOCUMENTATION.md](#).